Code Coverage Report

Project: Student Attendance Management System. Activity: Part 3 testing of completed Sprint 1 and Sprint 2 work. Execution date: 6 November 2026. Branch: `feature/testing-feedback`. Base commit: `438845cb25f6`. No Sprint 3 or application feature changes were made.

Measured result

Executable-line coverage is 94.89%: 3,270 of 3,446 executable statements were exercised. There are 176 unexecuted statements across the measured source. The scope contains 78 Python files, including empty package files.

Formula: covered executable statements / total executable statements x 100 = 3,270 / 3,446 x 100 = 94.89%.

This is instrumented execution evidence from coverage.py 7.17.2 with branch=True, collected during the final 530-test Django run (526 passed, four skipped). It is not an estimate from test counts. No required coverage percentage is specified in the attached deliverable instructions, so this report records the measured value without inventing an acceptance threshold.

Scope and exclusions

The measured source is `apps/` and `config/`, including unexecuted importable modules. Exclusions are migrations, test_*.py, tests.py and config/test_runner.py. Third-party packages, the verification harness, manage.py, HTML templates, CSS and JavaScript are outside this Python structural-coverage scope. No application error path was manually excluded; the coverage JSON reports zero excluded lines. Browser correctness is reported separately.

The working tree adds two validation tests, 30 coverage follow-up tests in five new test files, strengthens one browser assertion, and pins coverage.py 7.17.2. The application code is unchanged. Results and coverage in this report come from the final full rerun after the follow-up tests were added; an earlier run on the same day measured 92.89% line and 85.38% branch coverage before them. Coverage measures the automated Django run only; browser and performance results are separate evidence.

Results by component

Component	Lines hit	Statements	Coverage	Missing
apps/__init__.py	0	0	N/A	0
apps/academics	678	679	99.85%	1
apps/accounts	873	886	98.53%	13
apps/attendance	752	762	98.69%	10
apps/audit	29	30	96.67%	1
apps/backups	613	757	80.98%	144
apps/notifications	245	251	97.61%	6
config	80	81	98.77%	1

Largest line-coverage gaps

Source file	Missing statements	Line coverage
apps/backups/storage.py	131	43.29%
apps/backups/views.py	5	94.68%

Source file	Missing statements	Line coverage
apps/backups/services.py	4	97.45%
apps/accounts/otp.py	3	98.35%
apps/accounts/permissions.py	3	96.25%
apps/accounts/services.py	3	97.39%
apps/attendance/corrections.py	3	96.00%
apps/attendance/models.py	3	94.74%
apps/notifications/models.py	3	90.00%
apps/notifications/services.py	3	98.26%
apps/accounts/models.py	2	95.83%
apps/attendance/views.py	2	98.33%

`apps/backups/storage.py` accounts for 131 of the 176 unexecuted statements; 126 of them are native Windows access-control and reparse-point paths that cannot execute on this Mac. Measuring them requires the suite to run on Windows and the results to be combined. The 30 follow-up tests closed the earlier gaps in academics views and forms, two-factor pages, snapshot validation, attendance services, the scheduler command and the server entry points. Every other file now misses at most five statements. Unexecuted code is a test gap, not proof of a defect.

Complete per-file counts are in the rerun's `coverage.json` and `coverage-report.txt` in `artifacts/part3/rerun-2026-11-06/`. The HTML coverage site displays each source line, highlights misses, and provides function/class indexes. The coverage JSON retains the exact missing line numbers and branch arcs for every file.

Correct interpretation

Coverage.py's combined statement-and-branch value is **94.19%**. Because branch measurement is enabled, that combined percentage differs from the line-only 94.89% reported here. The Branch Coverage Report separately records 90.64% of branch outcomes.

Higher coverage alone does not establish correct assertions, requirements completeness, path coverage or production reliability. Four skipped checks contribute no new execution. Browser and performance runs were not merged into these coverage percentages.

Evidence and reproduction

Results of the final rerun (test outcomes, test log, coverage JSON and the HTML coverage pages) are kept locally in `artifacts/part3/rerun-2026-11-06/`. Logs and sources from the first run are retained in a separate local supporting-files archive. Neither is included in this submission folder.

Evidence: raw coverage JSON, coverage console report, per-file metrics and instrumented test outcomes. Open `html/index.html` in `artifacts/part3/rerun-2026-11-06/` for source annotations. The `.coveragerc` and pinned development dependency reproduce the measurement scope.

```
OTP_REQUIRED=True VERIFICATION_RESULTS=artifacts/part3/rerun-2026-11-06/regression.json \
  .venv/bin/python -m coverage run --data-file=artifacts/part3/rerun-2026-11-06/.coverage \
  manage.py test --noinput --testrunner verification.test_runner.Runner
.venv/bin/python -m coverage json --data-file=artifacts/part3/rerun-2026-11-06/.coverage \
  -o artifacts/part3/rerun-2026-11-06/coverage.json
.venv/bin/python -m coverage html --data-file=artifacts/part3/rerun-2026-11-06/.coverage \
  -d artifacts/part3/rerun-2026-11-06/html
.venv/bin/python -m coverage report --data-file=artifacts/part3/rerun-2026-11-06/.coverage
```

Configure the disposable test database and captured-email environment as described in the Unit Test Report. Retain the run date, revision, commands and coverage configuration with any rerun. Do not overwrite these dated evidence files when testing another revision.